

Automater 시스템 아키텍처 UML 가이드

Factory & Automator Layer 정적/동적 모델링

Robert C. Martin의 UML 철학에 기반한 교육용 교재

대상 독자: TDD 기반 풀스택 PHP 개발자 (UML 해석 가능)

코드베이스: automater-dev (Python 3.12, SQLAlchemy 2.0, Playwright)

이 문서는 시스템 엔지니어가 automater 코드베이스의 /factory와 /automator 레이어를 이해하기 위해 필요한 최소한의 UML 다이어그램과 설명을 담고 있습니다. Martin이 강조한 "최소주의" 원칙에 따라 핵심 의존성 구조와 실행 흐름만을 다룹니다.

목차

1. Martin의 UML 핵심 철학

- 1.1 Class Diagram: 의존성의 방향을 읽어라
- 1.2 Sequence Diagram: 코드를 시각화하되, 모든 것을 그리지 마라
- 1.3 이 교재에 적용한 원칙

2. 시스템 개요 — 2-Layer Architecture

- 2.1 Factory Layer의 책임
- 2.2 Automator Layer의 책임
- 2.3 계약 경계: PostingSpec

3. 정적 UML — Automator Layer

- 3.1 Port ABCs: 의존성 역전 경계
- 3.2 Block 계층 구조: 7개 frozen dataclass
- 3.3 Command Pattern: PostStep + BlogEditor
- 3.4 BlockHandler Strategy: Block → PostStep 변환
- 3.5 ContentBuilder + JobRunner 조립

4. 정적 UML — Factory Layer

- 4.1 JobBuilder: DB → PostingSpec 변환
- 4.2 BatchWorker: Batch 실행 오케스트레이션
- 4.3 Dispatcher + ComboGenerator

5. 동적 UML — Sequence Diagrams

- 5.1 JobRunner.run(): 핵심 실행 흐름
- 5.2 ContentBuilder.build(): 콘텐츠 생성 시퀀스
- 5.3 BatchWorker.run_batch(): 배치 처리 흐름

6. 테스트 코드로 검증하는 아키텍처

- 6.1 RecordingEditor로 본 실행 시퀀스
- 6.2 SimpleNameSpace Fake로 본 계약 검증
- 6.3 Port ABC 준수 검증

7. 부록: 새로운 Block 타입 추가 체크리스트

1. Martin의 UML 핵심 철학

Robert C. Martin은 UML을 코드를 대체하는 것이 아니라 코드의 지름길로 사용해야 한다고 강조합니다. 다이어그램에서 코드를 떠올릴 수 없다면 "공중에 성을 짓고 있는 것"이라 경고합니다. 이 장에서는 Class Diagram(3장)과 Sequence Diagram(4장)에서 추출한 핵심 원칙을 정리합니다.

1.1 Class Diagram: 의존성의 방향을 읽어라

UML의 모든 화살표는 소스 코드 의존성 방향을 가리킵니다. SalariedEmployee가 Employee를 상속하면 화살표는 Employee를 향합니다. 이 단순한 규칙만으로 시스템의 의존성 순환을 발견하고 추상 클래스가 구체 클래스에 의존하는 위반을 잡아낼 수 있습니다.

Martin은 Class Diagram에서 다음을 권장합니다:

- Interface를 반드시 표시하라. 어떤 클래스가 인터페이스이고 어떤 것이 구현인지 독자가 즉시 알아야 한다.
- 수직은 상속, 수평은 연관. 이 관례만으로 다이어그램의 의미가 명확해진다.
- 그룹핑하라. 관련 클래스를 시각적으로 묶고, 그룹 간 연결을 최소화하고 규칙적으로 유지하라.
- 모든 메서드를 나열하지 마라. 대표적인 메서드 몇 개로 독자에게 "아이디어"를 전달하면 충분하다.

1.2 Sequence Diagram: 코드를 시각화하되, 모든 것을 그리지 마라

Martin은 4장 첫 문장에서 단호하게 말합니다: 모든 메서드에 대해 시퀀스 다이어그램을 만들지 말라고. 시퀀스 다이어그램은 객체 그룹의 협력 방식을 설명해야 할 "즉각적인 필요"가 있을 때만 사용해야 합니다.

Martin의 원칙

"코드가 스스로 설명할 수 있다면, 다이어그램은 중복이며 낭비이다." 시퀀스 다이어그램은 한 페이지에 들어가야 하며, 설명 텍스트를 위한 충분한 여백이 남아야 합니다. 수십, 수백 개를 그리지 마세요. 공통 패턴에 집중하세요.

하나의 거대한 다이어그램 대신 여러 개의 작은 시나리오로 분리하라는 점도 핵심입니다. 고수준 다이어그램이 저수준보다 유용합니다. 시스템의 공통 주제와 관행을 노출하기 때문입니다.

1.3 Martin이 경고하는 안티패턴

Martin은 다이어그램 남용의 구체적인 안티패턴을 열거합니다. 이 코드베이스를 다룰 때 다음을 피해야 합니다:

- 프로세스가 요구하니까 그리는 것. 다이어그램은 커뮤니케이션 도구이지 산출물이 아닙니다.
- 코딩 전에 포괄적 설계 문서를 만드는 것. 이런 문서는 거의 가치가 없으며 막대한 시간을 소비합니다.
- 다른 사람이 코딩하도록 다이어그램을 그리는 것. 진정한 아키텍트는 자기가 만든 설계로 직접 코딩합니다.
- 모든 메서드에 시퀀스 다이어그램을 만드는 것. Martin은 Ch.4 첫 문장에서 이것을 "terrible waste of time"이라 단언합니다.

1.4 다이어그램을 그려야 할 때 vs 멈춰야 할 때

Martin은 다이어그램을 그려야 하는 정확한 시점을 다섯 가지로 정리합니다:

- 여러 명이 동시에 작업할 부분의 구조를 이해해야 할 때. 모두가 동의하면 멈춰라.
- 설계 방향에 대해 의견이 다를 때. 타임박스를 정하고 결정이 나면 다이어그램을 지워라.
- 설계 아이디어를 실험하고 싶을 때. 코드로 사고를 마칠 수 있으면 멈춰라.
- 코드 구조를 다른 사람(또는 자신)에게 설명할 때. 코드를 보는 것이 나오면 멈춰라.
- 프로젝트 말기에 고객이 문서를 요청할 때. 이때만 영구 문서를 만들라.

Martin의 원칙	이 교재의 적용
Interface를 반드시 표시	모든 Port ABC에 <<interface>> 스테레오타입 표시 (Fig 3-1, 3-4)
수직=상속, 수평=연관	모든 다이어그램에서 일관 적용
그룹핑 + 최소 연결	Factory / Contract / Automator 3개 존으로 분리 (Fig 2-1)
대표 메서드만 표시	각 ABC에 2-3개 메서드만 표시, 나머지는 코드 참조
시퀀스는 소수만, 고수준 우선	3개 시나리오, 모두 고수준 (Fig 5-1~5-3)
코드에서 다이어그램을 확인	6장에서 테스트 코드로 직접 검증

Table 1-1. Martin의 원칙과 이 교재에서의 적용 매핑

1.5 이 교재에 적용한 원칙

이 교재는 위 철학을 직접 적용합니다. 정적 다이어그램에서는 인터페이스(Port ABC)를 명시적으로 표시하고, 의존성 방향을 화살표로 일관되게 표현합니다. 동적 다이어그램은 3개의 핵심 시나리오만 선별했으며, 각각 한 페이지 이내로 유지합니다. Martin이 12인 팀의 백만 줄 프로젝트에 25-200페이지 문서를 권장한 것에 맞춰, 이 교재도 약 30페이지로 제한합니다.

2. 시스템 개요 — 2-Layer Architecture

automater 시스템은 /api, /factory, /automator 세 패키지로 구성됩니다. 이 교재에서는 /api를 제외하고, 핵심 비즈니스 로직인 factory와 automator만 다룹니다. 두 레이어는 PostingSpec이라는 불변 계약 객체를 통해 연결됩니다.



Fig 2-1. 시스템 2-Layer 개요 (DIP 적용)

2.1 Factory Layer의 책임

Factory는 데이터베이스에서 Campaign, Combination, Layout, Preset 등을 읽어 PostingSpec으로 조립합니다. 또한 BatchDispatcher가 미사용 Combination을 가용 Account에 배분하고, BatchWorker가 배치 단위로 실행을 관리합니다. Factory는 automator의 구체 구현에 의존하지 않으며, PostingSpec 계약과 Port ABC만 알고 있습니다.

2.2 Automator Layer의 책임

Automator는 PostingSpec을 받아 실제 블로그 포스팅을 실행합니다. SpecValidator로 사전 검증 후, ContentBuilder가 블록별로 텍스트 생성과 이미지 처리를 수행하고, JobRunner가 BlogEditor ABC를 통해 브라우저를 제어합니다. TextGenerator, ImageProcessor 같은 외부 의존성은 Port ABC로 역전되어 있어 테스트 시 StubTextGenerator, NoopImageProcessor로 대체됩니다.

2.3 계약 경계: PostingSpec

PostingSpec은 두 레이어의 유일한 접점입니다. frozen=True 데이터클래스로 불변이며, account, title, body(Section/Block 튜플), publish, setting 다섯 필드를 가집니다. PHP로 비유하면 readonly class와 같은 역할이며, 절대로 로직을 가지지 않는 순수 데이터 객체입니다.

```
# automator/contracts.py
@dataclass(frozen=True)
class PostingSpec:
    """Immutable posting specification."""
    account: AccountOption
    title: TitleOption = field(default_factory=TitleOption)
    body: tuple[Section, ...] = ()
    publish: PublishOption = field(default_factory=PublishOption)
    setting: RunSetting = field(default_factory=RunSetting)
```

PHP 비유: readonly class	PHP 8.2의 readonly class PostingSpec { public function __construct(public readonly AccountOption \$account, ...) {} } 와 구조적으로 동일합니다. frozen=True는 __setattr__을 금지하여 생성 후 어떤 필드도 변경할 수 없습니다. 이 불변성이 factory와 automator 사이의 계약을 안전하게 보장합니다.
----------------------------------	--

2.4 의존성 방향의 핵심: 양방향 화살표가 없다

Fig 2-1에서 가장 중요한 점은 화살표가 양쪽 레이어에서 모두 Contracts를 향한다는 것입니다. Factory는 PostingSpec을 '만들기 위해' contracts를 import하고, Automator는 PostingSpec을 '실행하기 위해' contracts를 import합니다. 하지만 Factory가 Automator를 직접 import하거나 그 반대는 절대 없습니다.

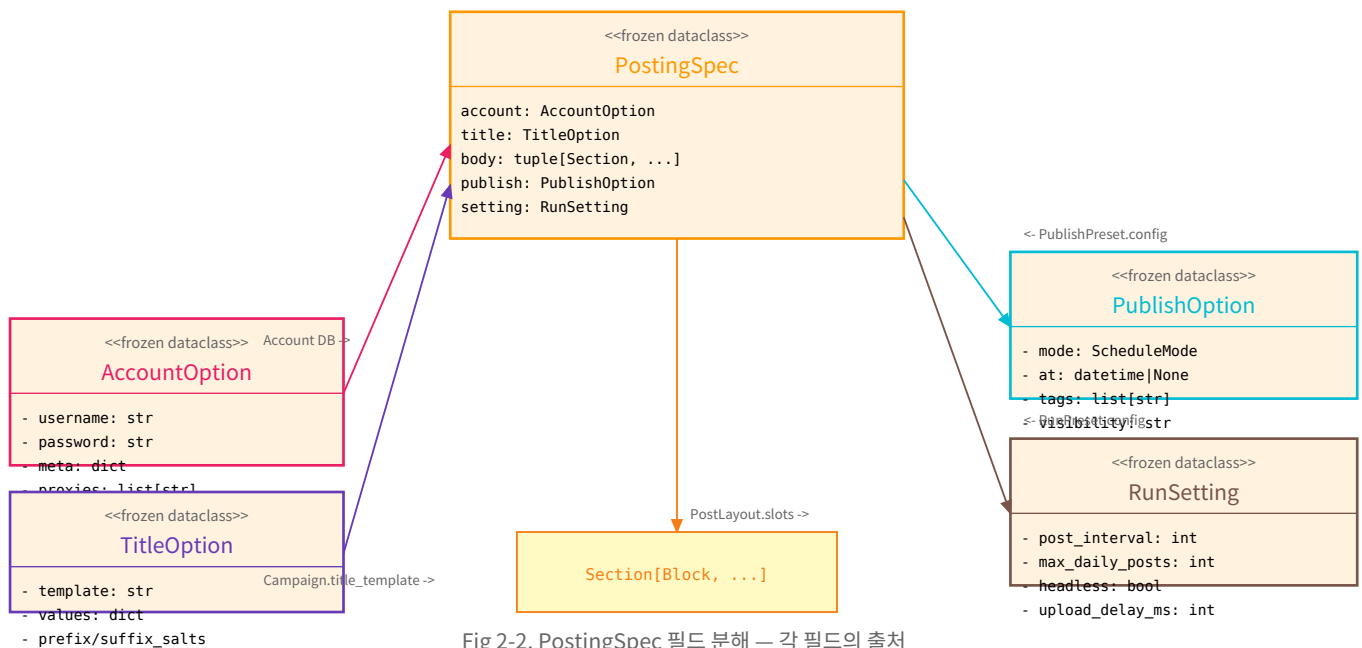
이 단방향 의존성은 Martin이 6장(OOD 원칙)에서 설명하는 DIP(Dependency Inversion Principle)의 교과서적 적용입니다. 고수준 정책(Factory의 배치 관리, Automator의 콘텐츠 생성)이 저수준 세부사항(구체 에디터, 구체 텍스트 생성기)에 의존하지 않고, 둘 다 추상(PostingSpec, Port ABC)에 의존합니다.

소스 모듈	import 대상	의존 방향
factory.job_builder	automator.contracts.PostingSpec	Factory -> Contracts
factory.job_builder	automator.block_factory.create_block	Factory -> Automator utils
factory.job_builder	automator.preset_loader.load_publish	Factory -> Automator utils
factory.batch_worker	automator.runner.JobRunner	Factory -> Automator
factory.batch_worker	automator.editor.BlogEditor (ABC)	Factory -> Contracts
automator.runner	automator.contracts.PostingSpec	Automator -> Contracts
automator.content_builder	automator.ports.TextGenerator (ABC)	Automator -> Ports
automator.block_handlers	automator.ports (TYPE_CHECKING only)	Automator -> Ports

Table 2-1. 주요 모듈 간 import 의존성 — 순환이 없음에 주목

2.5 PostingSpec 필드 분해

PostingSpec의 다섯 필드가 각각 어떤 정보를 담고, 어디서 생성되어 어디서 소비되는지를 분해합니다.



이 다이어그램에서 주목할 점은 모든 필드의 출처가 명확하게 분리되어 있다는 것입니다. account는 Account DB 테이블에서, title은 Campaign.title_template + Combination.keywords에서, body는 PostLayout.slots에서, publish/setting은 각각의 Preset.config JSON에서 옵니다. 이 분리 덕분에 각 필드를 독립적으로 테스트할 수 있습니다.

2.6 이 교재 읽기 가이드

이 교재는 순서대로 읽을 필요가 없습니다. 목적에 따라 다른 경로를 권장합니다:

- 새로운 Block 타입을 추가해야 할 때: 3.2 (Block 계층) -> 3.4 (BlockHandler) -> 7.1 (체크리스트) -> 6.4 (테스트).
- 배치 실행 흐름을 이해해야 할 때: 5.3 (BatchWorker 시퀀스) -> 4.2 (BatchWorker 정적) -> 5.1 (JobRunner 시퀀스).
- PostingSpec 계약을 이해해야 할 때: 2.3-2.5 (계약 경계) -> 4.1 (JobBuilder) -> 3.5 (ContentBuilder).
- 의존성 구조 전체를 파악해야 할 때: 2.1-2.4 (시스템 개요) -> 3.1 (Port ABCs) -> 7.1 (전체 의존성 요약).
- 테스트 전략을 이해해야 할 때: 6장 전체를 순서대로 읽고, 7.3 (SOLID 매핑)을 참조.

3. 정적 UML — Automator Layer

Automator 레이어는 DIP(의존성 역전 원칙)를 철저히 적용합니다. Martin이 3장에서 강조한 "인터페이스를 반드시 표시하라"는 원칙에 따라 Port ABC를 모든 다이어그램에서 명시합니다.

3.1 Port ABCs: 의존성 역전 경계

automator/ports.py는 세 개의 Abstract Base Class를 정의합니다. 이것은 Martin이 "경계를 표시하라"고 한 DIP의 핵심입니다. ContentBuilder와 BlockHandler는 이 ABC에만 의존하며, 구체 구현(GeminiGenerator, LocalImageProcessor)은 composition root에서 주입됩니다.

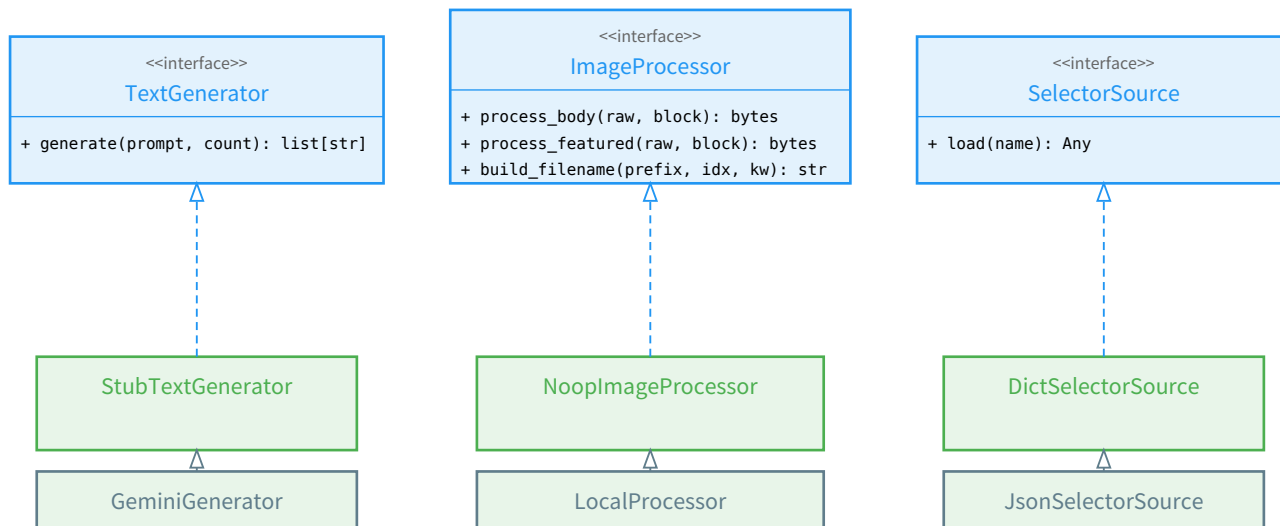


Fig 3-1. Port ABCs (<<interface>>)와 구현체 계층

PHP 비유

TextGenerator는 PHP의 interface TextGenerator { public function generate(string \$prompt, int \$count): array; } 와 동일합니다. Python에서는 ABC + @abstractmethod로 구현합니다. StubTextGenerator는 PHPUnit의 Mock과 유사하지만, 직접 작성된 test double입니다.

3.2 Block 계층 구조: 7개 frozen dataclass

automator/options.py는 7종의 Block 타입을 정의합니다. 모든 Block은 frozen=True로 불변이며, Python Union 타입으로 봉인(sealed)됩니다. Martin이 말한 "변수와 메서드를 모두 나열하지 말라"는 원칙을 따라, 각 Block의 대표 속성만 표시합니다.

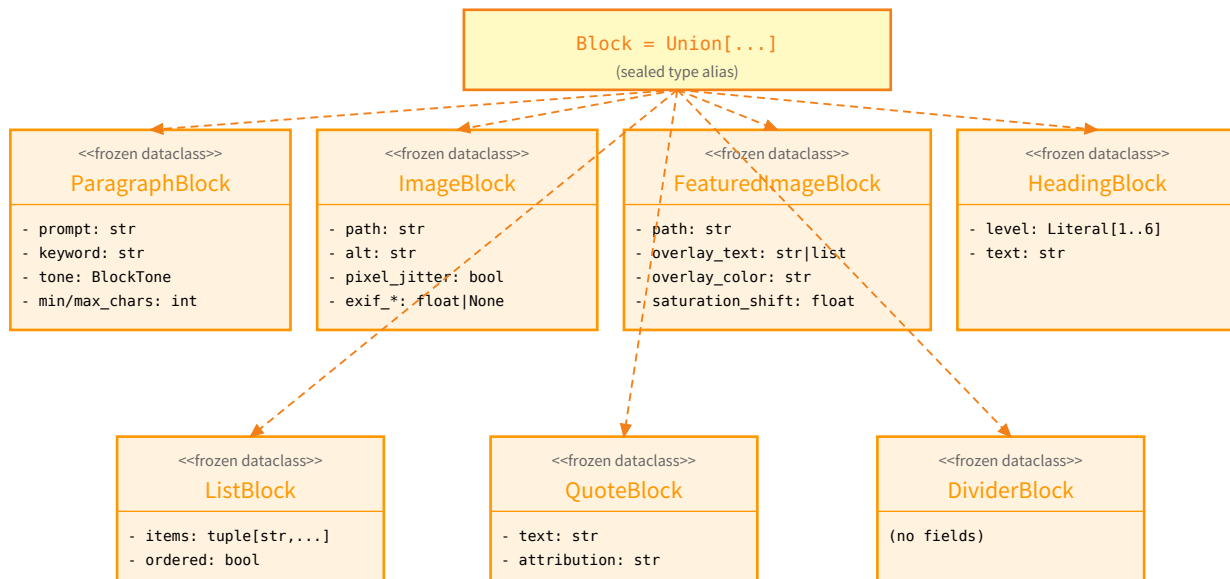


Fig 3-2. Block 타입 계층 (모든 타입은 frozen dataclass)

Section은 Block의 컨테이너입니다. role("intro", "body", "cta" 등)와 tone("informational", "review" 등)를 가지며, PostingSpec.body는 tuple[Section, ...]로 복수의 섹션을 담습니다.

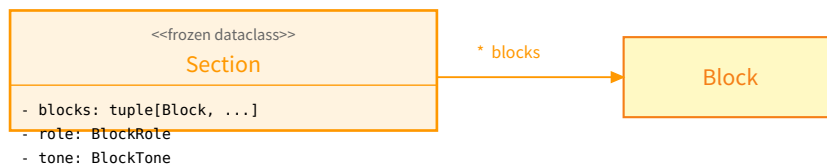


Fig 3-3. Section → Block 포함 관계

3.3 Command Pattern: PostStep + BlogEditor

Martin이 3장에서 보여준 ATM 예제처럼, automator는 행위를 "명령 객체"로 캡슐화합니다. PostStep ABC는 자기 자신을 실행하는 명령(self-executing command)이며, BlogEditor ABC의 7개 primitive를 조합하여 동작합니다. PHP의 Command Pattern과 동일한 구조입니다.

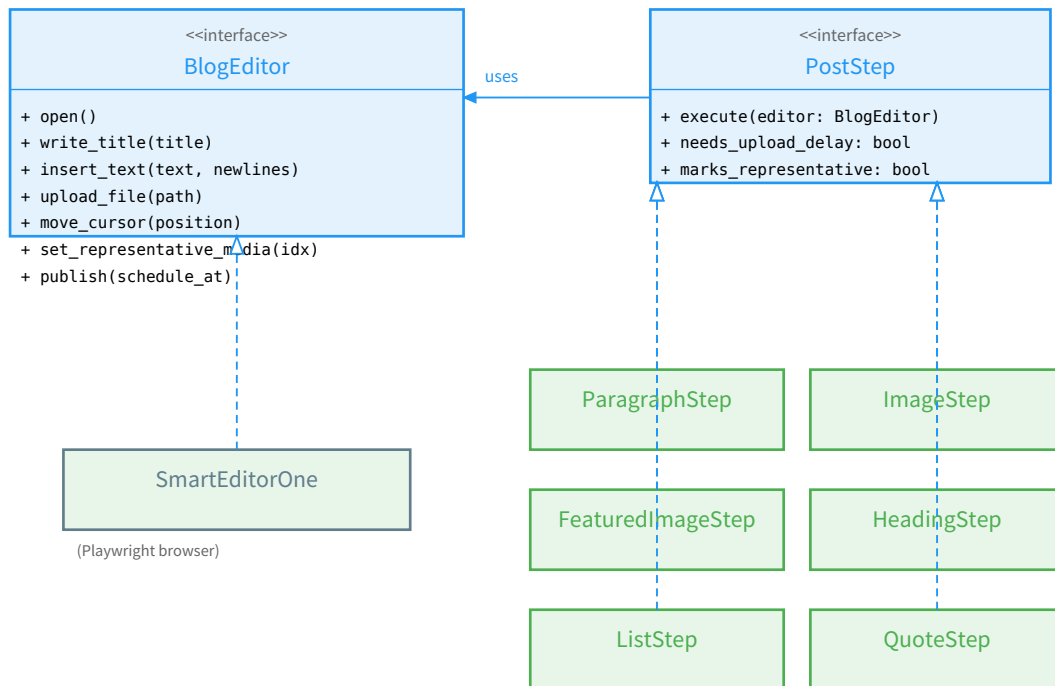


Fig 3-4. PostStep Command 계층과 BlogEditor ABC

각 **PostStep**의 `execute()`는 단 하나의 **BlogEditor** primitive를 호출합니다: **ParagraphStep** → `insert_text`, **ImageStep** → `upload_file`, **HeadingStep** → `insert_text`. 이는 Martin이 강조한 SRP(단일 책임 원칙)를 명령 수준에서 적용한 것입니다.

PHP 비유

BlogEditor는 PHP의 interface `BlogEditor { public function open(): void; ... }`와 같습니다. **PostStep**은 interface `PostStep { public function execute(BlogEditor $editor): void; }`로, Laravel의 Job/Command와 구조적으로 동일합니다.

3.4 BlockHandler Strategy: Block → PostStep 변환

BlockHandler는 Strategy 패턴으로 각 Block 타입을 PostStep으로 변환합니다. Martin의 3장에서 «delegates» 스테레오타입으로 표현한 위임 패턴과 유사합니다. HANDLERS 레지스트리는 type(block) → handler 매핑을 제공합니다.

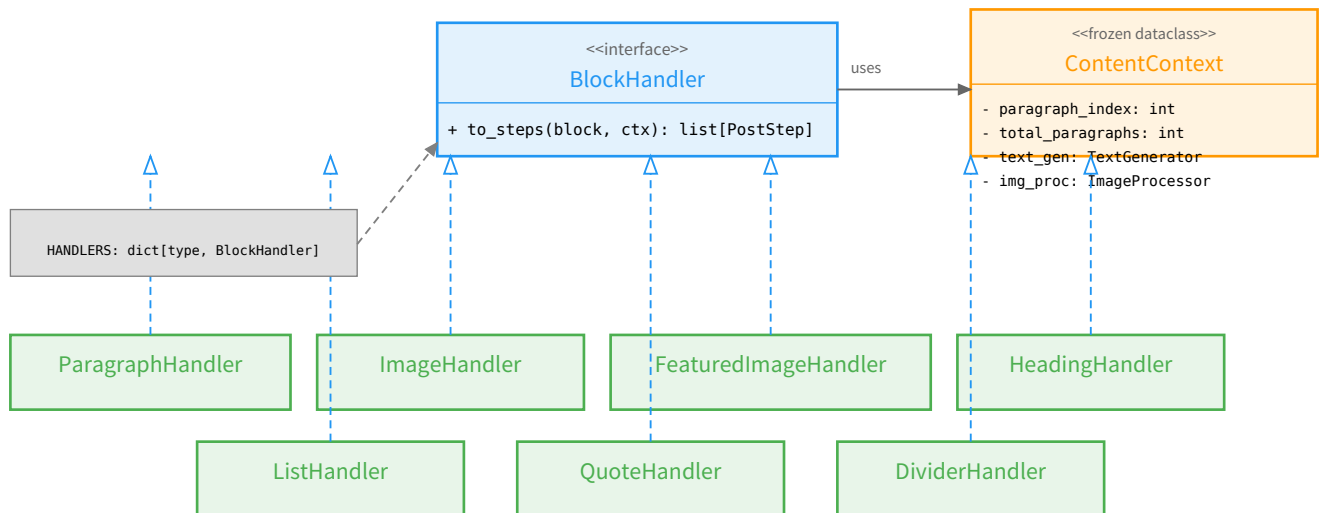


Fig 3-5. BlockHandler Strategy 패턴과 HANDLERS 레지스트리

각 Handler의 to_steps() 메서드는 Block을 하나 이상의 PostStep으로 변환합니다. 이 변환 규칙은 명확한 네이밍 컨벤션을 따릅니다:

Block 타입	Handler	생성되는 Step	BlogEditor primitive
ParagraphBlock	ParagraphHandler	ParagraphStep	insert_text()
ImageBlock	ImageHandler	ImageStep	upload_file()
FeaturedImageBlock	FeaturedImageHandler	FeaturedImageStep	upload_file()
HeadingBlock	HeadingHandler	HeadingStep	insert_text()
ListBlock	ListHandler	ListStep	insert_text()
QuoteBlock	QuoteHandler	QuoteStep	insert_text()
DividerBlock	DividerHandler	(none)	(no-op)

Table 3-1. XxxBlock -> XxxHandler -> XxxStep 네이밍 컨벤션

이 패턴의 핵심은 7개의 BlogEditor primitive만으로 모든 블록 타입을 처리한다는 점입니다. insert_text가 paragraphs, headings, lists, quotes를 모두 커버하고, upload_file이 images와 featured images를 커버합니다. 새 블록 타입이 추가되어도 BlogEditor에 새 메서드를 추가할 필요가 거의 없습니다. 이것은 Martin이 권장하는 ISP(Interface Segregation Principle)의 적용입니다.

ParagraphHandler의 to_steps() 구현을 보면 ContentContext를 통해 TextGenerator Port에 접근하는 방식을 확인할 수 있습니다:

```
class ParagraphHandler(BlockHandler):
    def to_steps(self, block: ParagraphBlock,
                ctx: ContentContext) -> list[PostStep]:
        prompt = build_prompt(
            block,
            paragraph_index=ctx.paragraph_index,
            total_paragraphs=ctx.total_paragraphs,
        )
        text = ctx.text_gen.generate(prompt, 1)[0]
        ctx.paragraph_index += 1
        return [ParagraphStep(text=text, newlines=block.newlines)]
```

**PHP 비유:
Strategy
Pattern**

PHP에서는 interface BlockHandler { public function toSteps(Block \$block, Context \$ctx): array; } 로 정의하고, HANDLERS는 [ParagraphBlock::class => new ParagraphHandler()] 형태의 associative array가 됩니다. Python의 dict[type, BlockHandler]과 동일한 구조입니다.

3.5 ContentBuilder + JobRunner 조립

JobRunner는 Mediator로서 SpecValidator와 ContentBuilder를 조율합니다. ContentBuilder는 PostingSpec을 받아 title 생성, 블록별 handler 호출, 스케줄 계산을 수행하고 _PostContent(내부 DTO)를 반환합니다. JobRunner._execute()는 이 DTO의 steps를 순서대로 BlogEditor에 실행합니다.

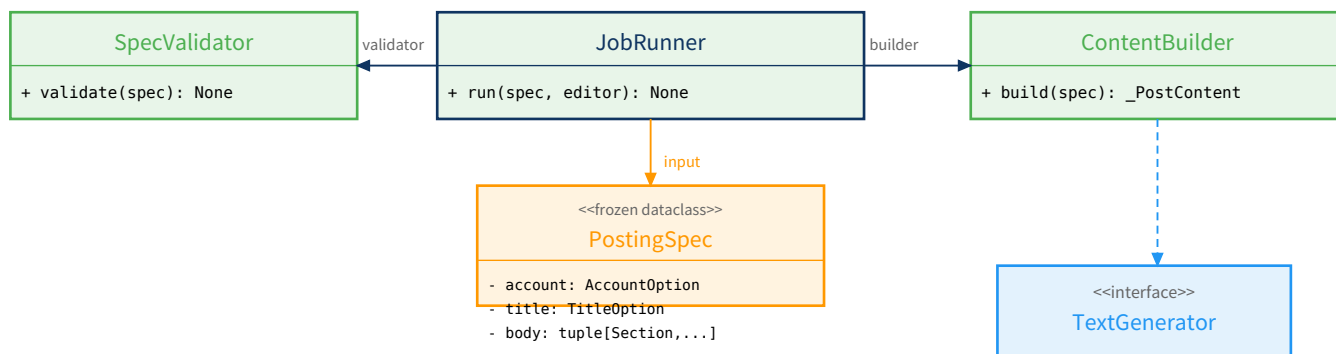


Fig 3-6. JobRunner 조립 — SpecValidator, ContentBuilder, Port ABCs

4. 정적 UML — Factory Layer

Factory 레이어는 DB 모델에서 PostingSpec을 조립하고, 배치를 관리합니다. Martin의 원칙에 따라 ORM 모델 자체는 생략하고, 조립/실행 로직의 의존성 구조만 표시합니다.

4.1 JobBuilder: DB → PostingSpec 변환

job_builder.py는 클래스가 아닌 순수 함수들의 집합입니다. build_posting_spec()이 진입점이며, 내부적으로 _build_title, _build_body_from_layout, _build_body_default를 호출합니다. Martin의 <> 스테레오타입에 해당합니다.

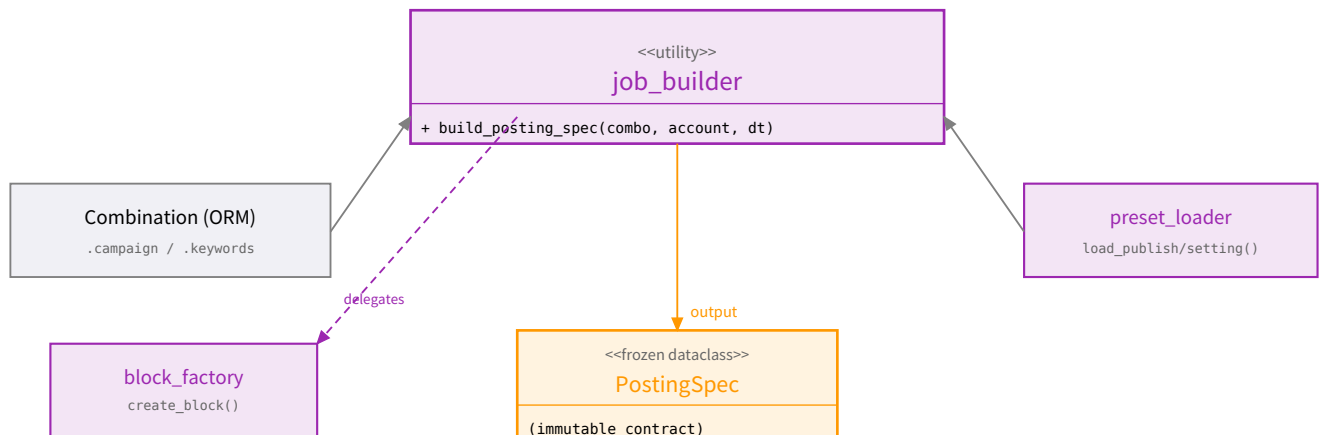


Fig 4-1. JobBuilder 함수 의존성 구조

build_posting_spec()의 내부 흐름은 다음과 같습니다:

- `_build_values(combo)`: Combination의 keywords를 category slug -> value dict로 변환. 비활성 Affix는 자동으로 제거됩니다 (예: '강남구'에서 inactive suffix '구' 제거 -> '강남').
- `_build_title(combo)`: Campaign.title_template에 values를 대입하여 TitleOption 생성.
- `_build_body_from_layout(layout, combo)`: PostLayout.slots를 순회하며 block_factory.create_block()으로 각 LayoutSlot을 Block dataclass로 변환. 이때 {keyword}, {region} 같은 placeholder가 보간됩니다.
- `preset_loader.load_publish/load_setting`: Preset JSON config를 PublishOption/RunSetting frozen dataclass로 역직렬화.

media_resolver는 Callable[[int], str] 타입의 콜백으로, LayoutSlot config의 media_id(int)를 실제 파일 경로(str)로 변환합니다. factory/media_resolver.py의 build_media_resolver()가 이 콜백을 생성하며, 내부적으로 DB 조회 + LocalStorage.resolve()를 수행합니다.

4.2 BatchWorker: Batch 실행 오케스트레이션

BatchWorker는 Batch ORM 객체를 받아 각 BatchItem을 순서대로 처리합니다. 핵심 의존성은 JobRunner와 EditorFactory(Callable[[Account], BlogEditor])입니다. Martin의 Ch.2 패턴처럼, Worker는 BlogEditor ABC에 의존하며 구체 에디터(SmartEditorOne)는 factory 함수로 주입됩니다.

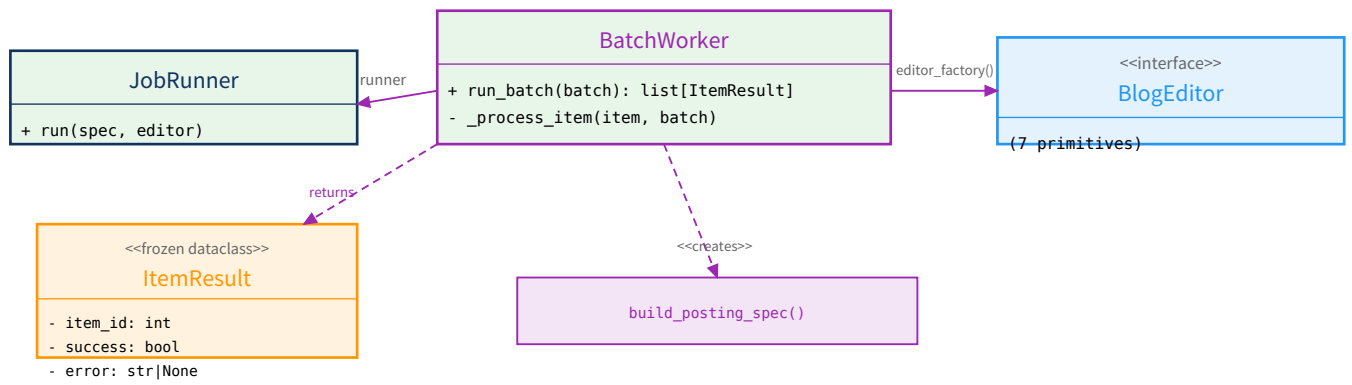


Fig 4-2. BatchWorker 의존성과 실행 결과

4.3 Dispatcher + ComboGenerator

BatchDispatcher는 미사용 Combination을 가용 Account에 최대 40개씩 배분합니다. ComboGenerator는 Campaign의 CampaignSlot별 키워드를 카르테시안 곱하여 Combination 행을 생성합니다. build_combos()는 순수 함수로 DB 없이 테스트 가능합니다.

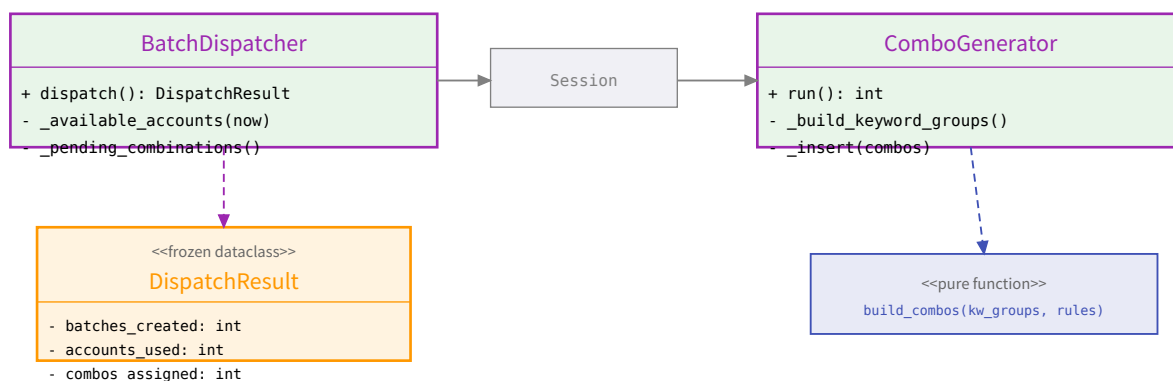


Fig 4-3. BatchDispatcher와 ComboGenerator

테스트 가능성

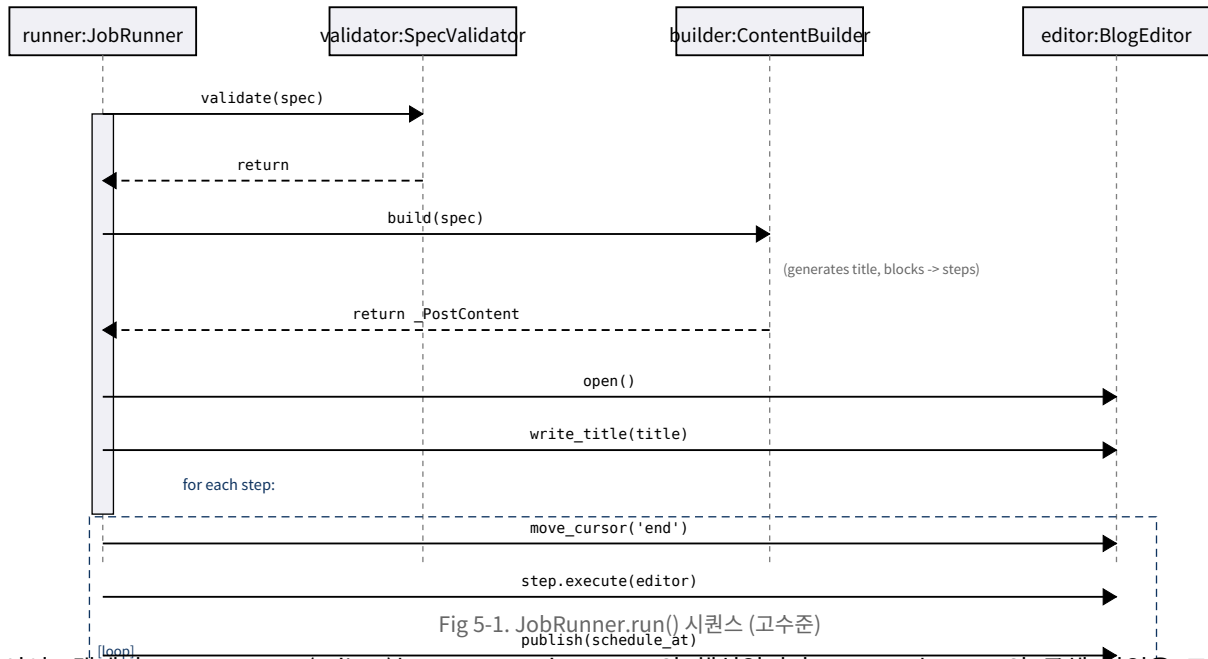
build_combos()는 순수 함수입니다. 입력은 int 리스트의 리스트와 spacing_rule_id 리스트, 출력은 ComboSpec 리스트입니다. DB 세션 없이 단위 테스트가 가능합니다. 이 분리는 Martin이 권장하는 "구조를 테스트하기 쉽게 만들어라"는 원칙의 직접 적용입니다.

5. 동적 UML — Sequence Diagrams

Martin의 원칙에 따라, 이 장에서는 시스템의 핵심 시나리오 3개만 시퀀스 다이어그램으로 표현합니다. "수십 개를 그리지 말라. 공통 패턴에 집중하라." 각 다이어그램은 고수준 관점을 우선하며, 저수준 알고리즘은 코드 참조를 안내합니다.

5.1 JobRunner.run(): 핵심 실행 흐름

이 시퀀스는 시스템에서 가장 중요한 흐름입니다. 하나의 PostingSpec이 어떻게 검증, 콘텐츠 생성, 에디터 실행을 거쳐 발행되는지를 보여줍니다.



이 다이어그램에서 `step.execute(editor)`는 Command Pattern의 핵심입니다. Runner는 step의 구체 타입을 모릅니다. instanceof/switch가 아닌 다형성으로 동작합니다. Martin이 4장에서 강조한 "인터페이스로 메시지를 보내라"의 직접 적용입니다.

이 시퀀스에서 특히 주목해야 할 세 가지 설계 결정이 있습니다:

- `validate` → `build` → `execute` 삼단 분리. `SpecValidator`가 실패하면 `ContentBuilder`는 실행되지 않습니다. `ContentBuilder`가 실패하면 `BlogEditor`는 열리지 않습니다. 이것은 fail-fast 원칙이며, 비싼 외부 자원(브라우저)을 불필요하게 소비하지 않습니다.
- `step.execute(editor)` 다형 디스패치. `JobRunner._execute()`는 for 루프에서 각 step의 `execute()`를 호출합니다. `ParagraphStep`이면 `insert_text`가, `ImageStep`이면 `upload_file`가 호출됩니다. PHP의 polymorphic dispatch (`$step->execute($editor)`)와 동일합니다.
- finally 블록의 임시 파일 정리. `ImageHandler`가 생성한 임시 파일은 다이어그램에 표시하지 않았지만, 실행이 성공하든 실패하든 항상 정리됩니다. Martin의 원칙대로 이 세부사항은 시퀀스 다이어그램이 아닌 코드에서 확인해야 합니다.

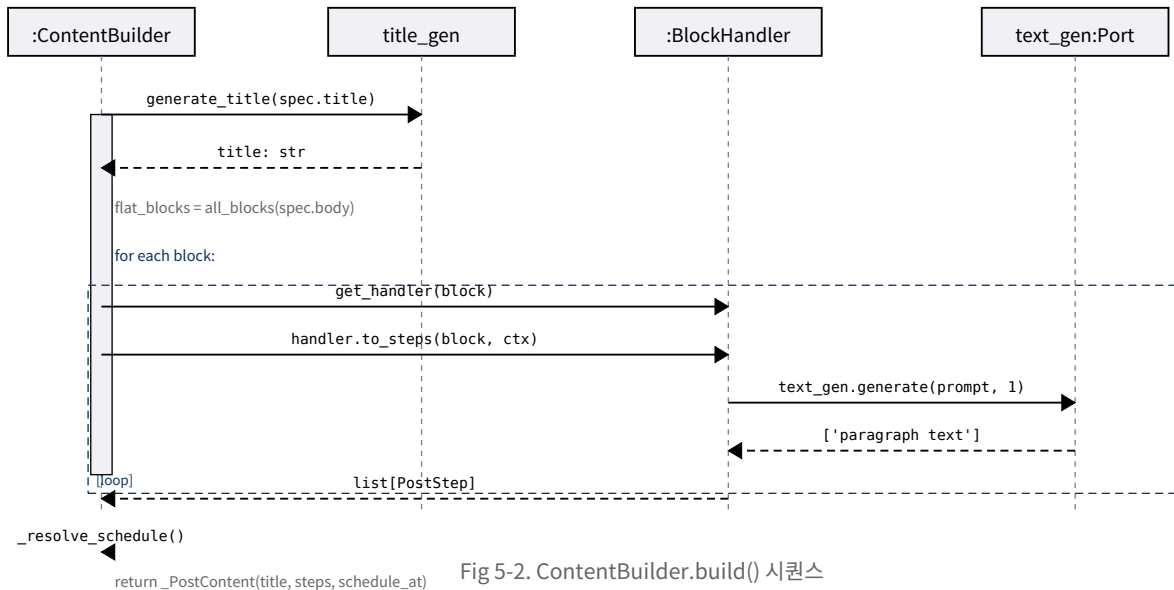
Martin이 4장에서 제시한 "코드가 충분히 표현적이라면 다이어그램은 불필요하다"는 관점에서, `JobRunner.run()`의 실제 코드를 확인합니다. 다이어그램과 코드가 어떻게 1:1로 대응하는지 주목하세요:

```

# automator/runner.py - JobRunner.run()
def run(self, spec: PostingSpec, editor: BlogEditor) -> None:
    self._validator.validate(spec) # Fig 5-1: validate(spec)
    post = self._builder.build(spec) # Fig 5-1: build(spec)
    self._execute(editor, post, spec) # Fig 5-1: open/write/loop/publish
    
```

5.2 ContentBuilder.build(): 콘텐츠 생성 시퀀스

ContentBuilder가 PostingSpec을 _PostContent로 변환하는 내부 흐름입니다. 이 시퀀스는 BlockHandler Strategy 패턴이 동적으로 어떻게 작동하는지 보여줍니다.

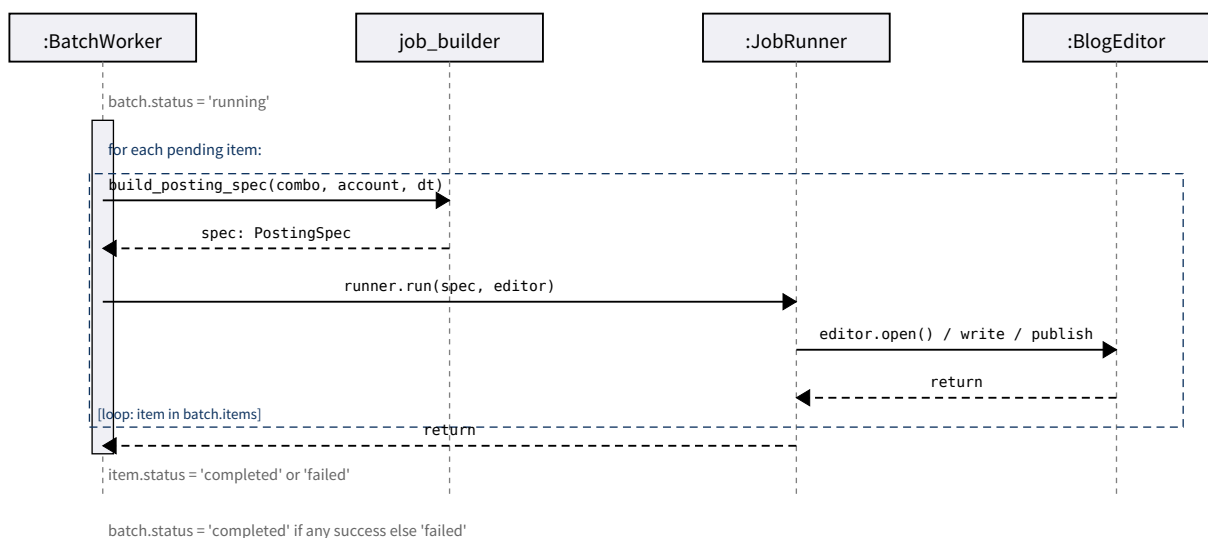


이 시퀀스에서 핵심은 `get_handler(block)` 호출입니다. 이것은 HANDLERS 레지스트리에서 block의 타입에 해당하는 handler를 조회하는 것으로, Fig 3-5의 Strategy 패턴이 동적으로 작동하는 순간입니다. ContentBuilder는 block의 구체 타입을 알 필요 없이, `handler.to_steps()`만 호출하면 됩니다.

또한 ContentContext가 handler 간에 공유되어 paragraph_index를 증가시키는 점에 주목하세요. 이것은 SEO 프롬프트 생성 시 단락의 위치(첫 번째/중간/마지막)에 따라 키워드 배치 전략이 달라지기 때문입니다. PHP에서는 Context 객체를 참조로 전달하는 것과 동일합니다.

5.3 BatchWorker.run_batch(): 배치 처리 흐름

Factory 레이어의 최상위 실행 흐름입니다. BatchWorker가 각 BatchItem을 순회하며, 개별 실패가 전체 배치를 중단하지 않음을 보여줍니다.



이 시퀀스의 핵심 설계 결정은 개별 실패가 배치를 중단하지 않는다는 것입니다. `_process_item()`은 try/except로 모든 예외를 캐치하고 ItemResult를 반환합니다. 하나의 item이 실패해도 나머지 item은 계속 처리됩니다. 배치 전체의 status는 성공이 하나라도 있으면 'completed', 전부 실패하면 'failed'입니다.

또한 `build_posting_spec()`이 매 item마다 호출되는 점에 주목하세요. 같은 Campaign의 item이라도 각각의 Combination이 다른 키워드를 가지므로, PostingSpec은 매번 새로 생성됩니다. 이것은 PostingSpec의 불변성(`frozen=True`)과 함께 각 item의 독립성을 보장하는 설계입니다.

**Martin의 4장
관점에서**

Martin은 "고수준 다이어그램이 저수준보다 유용하다"고 강조합니다. Fig 5-3은 의도적으로 에디터 내부 동작(Fig 5-1의 step loop)을 숨기고, 배치 관리 흐름만 보여줍니다. 이 두 시퀀스가 서로 보완하여 시스템의 전체 그림을 완성합니다.

6. 테스트 코드로 검증하는 아키텍처

Martin은 "다이어그램에서 코드를 떠올릴 수 있어야 한다"고 강조합니다. 이 장에서는 위 UML 다이어그램이 클라이언트에게 어떤 인터페이스로 보이는지를 실제 테스트 코드로 검증합니다. PHP의 PHPUnit과 구조적으로 대응시키며 설명합니다.

6.1 RecordingEditor로 본 실행 시퀀스

Fig 5-1의 JobRunner.run() 시퀀스가 실제로 어떻게 검증되는지 봅니다. RecordingEditor는 BlogEditor ABC를 구현한 test double로, 모든 primitive 호출을 순서대로 기록합니다.

```
# tests/integration/automator/test_runner.py

class _RecordingEditor(BlogEditor):
    """Captures all primitive calls in order."""
    def __init__(self):
        self.calls: list[tuple] = []

    def open(self):
        self.calls.append(("open",))

    def write_title(self, title):
        self.calls.append(("title", title))

    def insert_text(self, text, newlines=2):
        self.calls.append(("insert_text", text, newlines))

    def upload_file(self, path):
        self.calls.append(("upload_file", path))

    # ... move_cursor, set_representative_media, publish ...
```

이 RecordingEditor는 PHP에서 Mock 객체의 `->expects($this->once())` 대신, 명시적으로 호출 순서를 기록하는 Spy 패턴입니다. 다음 테스트는 Fig 5-1의 시퀀스를 직접 검증합니다:

```
def test_run_calls_open_title_publish(runner, rec):
    """Minimum run: open -> title -> content -> publish."""
    runner.run(_spec(), rec)
    actions = [c[0] for c in rec.calls]
    assert actions[0] == "open" # Fig 5-1: first message
    assert actions[1] == "title" # Fig 5-1: write_title
    assert actions[-1] == "publish" # Fig 5-1: last message

def test_run_with_paragraphs(runner, rec):
    """ParagraphBlocks produce insert_text calls."""
    spec = _spec(body=(Section(blocks=(
        ParagraphBlock(prompt="A"),
        ParagraphBlock(prompt="B"),
    ))))
    runner.run(spec, rec)
    inserts = [c for c in rec.calls if c[0] == "insert_text"]
    assert len(inserts) == 2 # Fig 5-1: loop executes twice
```

TDD 포인트

이 테스트들은 Fig 5-1 시퀀스 다이어그램의 "실행 가능한 명세"입니다. 다이어그램이 변경되면 테스트도 변경되어야 합니다. PHP 개발자는 이것을 PHPUnit의 `assertSame` 체인으로 생각하면 됩니다.

6.2 SimpleNamespace Fake로 본 계약 검증

Fig 4-1의 JobBuilder 다이어그램이 실제로 어떻게 DB 없이 테스트되는지 봅니다. Python의 SimpleNamespace를 사용해 ORM 객체를 가볍게 흉내냅니다.

```
# tests/unit/factory/test_job_builder.py

def _keyword(slug, value, cat_id=1, affixes=None):
    category = SimpleNamespace(slug=slug, id=cat_id)
    return SimpleNamespace(
        category=category, value=value, affixes=affixes or []
    )

def _combo(keywords, title_template="{region} {subject}",
            layout=None, publish_preset=None, run_preset=None):
    campaign = SimpleNamespace(
        title_template=title_template,
        layout=layout,
        publish_preset=publish_preset,
        run_preset=run_preset,
    )
    return SimpleNamespace(keywords=keywords, campaign=campaign)
```

이 Fake 객체들로 build_posting_spec()의 전체 파이프라인을 검증합니다:

```
class TestBuildPostingSpec:

    def test_with_layout(self):
        """Fig 4-1: layout -> body blocks conversion."""
        layout = _layout(
            _slot(0, "heading", {"level": 2, "text": "{keyword}"}),
            _slot(1, "paragraph", {"keyword": "{keyword}"}),
        )
        combo = _combo(
            keywords=[_keyword("region", "gangnam", cat_id=1)],
            layout=layout,
        )
        spec = build_posting_spec(combo, _account(), scheduled)
        assert len(spec.body) == 1
        assert len(spec.body[0].blocks) == 2
        assert isinstance(spec.body[0].blocks[0], HeadingBlock)

    def test_image_media_id_resolved_to_path(self):
        """Fig 4-1: media_resolver callback in action."""
        resolver = lambda mid: f"/uploads/{mid}/photo.jpg"
        block = create_block("image", {"media_id": 42},
                              media_resolver=resolver)
        assert block.path == "/uploads/42/photo.jpg"
```

6.3 Port ABC 준수 검증

Fig 3-1의 Port ABC 다이어그램이 런타임에서도 올바른지 isinstance로 검증합니다. 이것은 Martin이 말한 "코드에서 다이어그램을 확인하라"의 가장 직접적인 형태입니다.

```
# tests/unit/automator/test_ports.py

def test_stub_text_gen_is_text_generator():
    """Fig 3-1: StubTextGenerator implements TextGenerator."""
    assert isinstance(StubTextGenerator(), TextGenerator)

def test_noop_img_proc_is_image_processor():
    """Fig 3-1: NoopImageProcessor implements ImageProcessor."""
    assert isinstance(NoopImageProcessor(), ImageProcessor)

def test_noop_returns_input_unchanged():
```

```

"""Noop double returns bytes unchanged (no side effects)."""
proc = NoopImageProcessor()
data = b"jpeg bytes"
assert proc.process_body(data, None) is data # identity
assert proc.process_featured(data, None) is data

```

PHP 비유: PHP에서는 interface 자체를 테스트하지 않지만, Mock이 interface를 제대로 구현하는지 PHPUnit의 createMock(TextGenerator::class)으로 보장합니다. Python의 isinstance(stub, ABC) 검증은 이와 동일한 안전망입니다.

6.4 BlockFactory 테스트: Fig 3-2 Block 계층 검증

Fig 3-2의 Block 타입 계층이 실제로 올바르게 동작하는지 BlockFactory 테스트로 검증합니다. create_block()은 문자열 키에서 적절한 frozen dataclass를 생성하며, placeholder 보간(interpolation)도 처리합니다.

```

# tests/unit/automator/test_block_factory.py

def test_all_block_types_registered():
    """Fig 3-2: Every concrete Block has a factory entry."""
    assert "paragraph" in FACTORIES
    assert "image" in FACTORIES
    assert "featured" in FACTORIES
    assert "heading" in FACTORIES
    assert "list" in FACTORIES
    assert "quote" in FACTORIES
    assert "divider" in FACTORIES

def test_keyword_placeholder():
    """{keyword} in config -> full keyword string."""
    block = create_block(
        "paragraph",
        {"keyword": "{keyword}", "tone": "review"},
        values={"region": "gangnam", "subject": "math"},
        keyword="gangnam math",
    )
    assert block.keyword == "gangnam math"

def test_image_media_id_resolved_to_path():
    """media_id in config -> path via resolver."""
    resolver = lambda mid: f"/uploads/{mid}/photo.jpg"
    block = create_block("image", {"media_id": 42},
        media_resolver=resolver)
    assert block.path == "/uploads/42/photo.jpg"

```

이 테스트는 Fig 4-1의 block_factory가 LayoutSlot의 JSON config를 Block dataclass로 정확히 변환하는지 검증합니다. 특히 media_id 해석과 placeholder 보간이라는 두 가지 핵심 변환 규칙이 올바른지 확인합니다.

6.5 SpecValidator 테스트: 방어적 계약 검증

Fig 5-1 시퀀스에서 첫 번째로 실행되는 validate(spec)이 실제로 어떤 조건을 검사하는지 테스트 코드로 확인합니다. SpecValidator는 Stateless이며, account, title, body layout, publish schedule, run setting 다섯 영역을 검증합니다.

```

# SpecValidator 00 00 (0000 00)
#
# _validate_account: username/password 0 0 00
# _validate_title: template 00 00
# validate_sections: FeaturedImageBlock 00 10
# _validate_publish: min_tags <= max_tags,
# ratio 00 0-100,
# schedule 00 00
# _validate_setting: post_interval >= 0,

```

```
# max_daily_posts >= 1
```

이 검증은 fail-fast 전략의 핵심입니다. 브라우저를 열기 전에 모든 설정 오류를 잡아내므로, Playwright 세션이나 Gemini API 호출 같은 비싼 자원을 불필요하게 소비하지 않습니다.

6.6 테스트 아키텍처 요약

이 코드베이스의 테스트는 Martin의 "코드에서 다이어그램을 확인하라"는 원칙을 세 가지 층위로 적용합니다:

층위	테스트 타입	검증 대상	사용하는 Test Double
Unit	test_block_factory.py	Block 생성 규칙 (Fig 3-2)	없음 (순수 함수)
Unit	test_job_builder.py	DB -> PostingSpec 변환 (Fig 4-1)	SimpleNamespace Fake
Unit	test_ports.py	Port ABC 준수 (Fig 3-1)	Stub/Noop doubles
Unit	test_title_generator.py	제목 생성 규칙	없음 (순수 함수)
Unit	test_seo_prompt.py	SEO 프롬프트 생성	없음 (순수 함수)
Integration	test_runner.py	실행 시퀀스 (Fig 5-1)	RecordingEditor + Stubs
Integration	test_content_builder.py	콘텐츠 생성 (Fig 5-2)	StubTextGenerator + Noop
Integration	test_batch_dispatcher.py	배치 배분 (Fig 4-3)	In-memory SQLite

Table 6-1. 테스트 층위별 검증 대상과 Test Double 매핑

TDD 포인트: Test Double 전략

이 코드베이스는 mock.patch를 거의 사용하지 않습니다. 대신 ports.py의 ABC를 구현한 전용 test double(StubTextGenerator, NoopImageProcessor, RecordingEditor)을 사용합니다. 이것은 Martin이 권장하는 의존성 주입의 자연스러운 결과입니다. Port ABC가 있으면 mock.patch 없이도 test double을 주입할 수 있기 때문입니다. PHP에서도 constructor injection + interface를 쓰면 Mockery 없이 테스트가 가능한 것과 동일합니다.

7. 부록: 새로운 Block 타입 추가 체크리스트

이 체크리스트는 위 UML 다이어그램을 실무에서 직접 활용하는 방법입니다. 새로운 Block 타입(예: VideoBlock)을 추가할 때, 각 다이어그램의 어느 부분이 영향을 받는지 추적합니다.

단계	파일	관련 다이어그램	작업
1	automator/options.py	Fig 3-2	VideoBlock frozen dataclass 정의 Block Union에 추가
2	automator/editor.py	Fig 3-4	새 primitive가 필요한 경우만 BlogEditor에 추가 (upload_file 재사용 가능하면 생략)
3	automator/editor.py	Fig 3-4	VideoStep(PostStep) 정의 execute()에서 BlogEditor primitive 호출
4	automator/block_handlers.py	Fig 3-5	VideoHandler(BlockHandler) 구현 HANDLERS 레지스트리에 등록
5	automator/block_factory.py	Fig 4-1	FACTORIES["video"] = VideoBlock 등록
6	tests/unit/	Fig 6-*	test_block_factory.py에 VideoBlock 테스트 test_ports.py에 handler 테스트

이 체크리스트의 핵심은 OCP(개방-폐쇄 원칙)의 적용입니다. 기존 코드를 수정하는 곳은 FACTORIES와 HANDLERS 레지스트리 등록뿐이고, 나머지는 모두 새 코드를 추가하는 것입니다. BlockHandler ABC와 PostStep ABC가 확장 지점을 제공하므로, ContentBuilder와 JobRunner는 전혀 수정하지 않습니다.

7.2 적용된 Design Pattern 정리

이 코드베이스에 적용된 핵심 디자인 패턴을 표로 정리합니다. 각 패턴이 어느 다이어그램에서 확인 가능한지, 그리고 PHP에서의 대응 구조를 함께 기록합니다.

Pattern	적용 위치	관련 Figure	PHP 대응
Command	PostStep + BlogEditor	Fig 3-4, 5-1	interface Command { execute(); }
Strategy	BlockHandler + HANDLERS registry	Fig 3-5	interface Handler + \$handlers[] array
Factory Method	block_factory.create_block() FACTORIES registry	Fig 4-1	\$factories[] + create(string \$type)
Template Method	JobRunner.run() validate -> build -> execute	Fig 5-1	abstract class Pipeline with ordered steps
Adapter	BatchWorker._build_ account_option()	Fig 4-2	Account ORM -> AccountOption VO
Dependency Injection	Port ABCs injected at composition root	Fig 3-1, 3-6	Constructor injection via container
Spy/Recording	RecordingEditor in tests	Fig 5-1 (검증)	PHPUnit spy mock
Null Object	DividerHandler returns []	Fig 3-5	NullHandler returning empty

Table 7-2. 적용된 디자인 패턴 — GoF + DI + Testing Patterns

7.3 SOLID 원칙 매핑

Martin의 UML 철학은 그의 SOLID 원칙과 직접 연결됩니다. 이 코드베이스에서 각 원칙이 어떻게 적용되었는지 정리합니다.

원칙	약자	코드베이스 적용
Single Responsibility	SRP	각 BlockHandler는 하나의 Block 타입만 처리. SpecValidator는 검증만, ContentBuilder는 생성만, JobRunner는 조율만.
Open/Closed	OCP	새 Block 추가 시 기존 ContentBuilder/JobRunner를 수정하지 않고 새 Handler와 Step을 추가하고 레지스트리에 등록만 함.
Liskov Substitution	LSP	모든 BlockHandler는 to_steps()의 계약을 준수. 모든 PostStep은 execute(editor)를 동일한 방식으로 호출 가능.
Interface Segregation	ISP	BlogEditor는 7개의 작은 primitive만 노출. TextGenerator, ImageProcessor는 각각 독립적인 ABC.
Dependency Inversion	DIP	Factory와 Automater 모두 Contracts/ABCs에 의존. 구체 구현(Gemini, Playwright)은 composition root에서 주입.

Table 7-3. SOLID 원칙 적용 매핑

7.4 용어 사전 (Glossary)

이 교재에서 사용된 핵심 용어를 Python/PHP 양쪽 관점에서 정리합니다.

용어	정의	Python 구현	PHP 대응
ABC	Abstract Base Class. 추상 메서드를 강제하는 기본 클래스	abc.ABC + @abstractmethod	interface 또는 abstract class
Port	의존성 역전 경계의 추상. 외부 서비스와의 접점	ports.py의 ABC	interface in app/Contracts/
Frozen Dataclass	생성 후 변경 불가능한 순수 데이터 객체	@dataclass(frozen=True)	readonly class (PHP 8.2+)
Test Double	테스트용 가짜 구현체의 총칭	stubs.py의 Stub/Noop 클래스	PHPUnit Mock or custom Fake
Composition Root	모든 의존성을 조립하는 진입점	main.py 또는 app factory	ServiceProvider or Container
Registry	타입 → 핸들러 매핑. 동적 디스패치의 구현	dict[type, Handler]	associative array or Container
Combination	키워드 카르테시안 곱의 한 행. 하나의 포스트에 대응	Combination ORM model	Eloquent Model or DTO
BatchItem	배치 내 하나의 작업 단위. Combination을 참조	BatchItem ORM model	Queue Job or Task

Table 7-4. 용어 사전 — Python/PHP 양쪽 관점

7.5 파일 레퍼런스

이 교재에서 다루는 핵심 파일 목록과 각 파일의 역할, 관련 다이어그램을 정리합니다.

파일 경로	역할	관련 Figure
automator/contracts.py	PostingSpec 계약 정의	Fig 2-1
automator/ports.py	TextGenerator, ImageProcessor, SelectorSource ABC	Fig 3-1
automator/options.py	Block 7종 + Section + PublishOption + RunSetting	Fig 3-2, 3-3
automator/editor.py	BlogEditor ABC + PostStep hierarchy	Fig 3-4
automator/block_handlers.py	BlockHandler ABC + 7 concrete handlers	Fig 3-5
automator/content_builder.py	PostingSpec -> _PostContent 변환	Fig 3-6, 5-2
automator/runner.py	JobRunner 오케스트레이터	Fig 3-6, 5-1
automator/block_factory.py	block_type string -> Block dataclass	Fig 4-1
automator/stubs.py	Test doubles (Stub, Noop, Dict)	Fig 3-1
factory/job_builder.py	Combination ORM -> PostingSpec	Fig 4-1
factory/batch_worker.py	배치 실행 관리	Fig 4-2, 5-3
factory/dispatcher.py	Combination -> Account 배분	Fig 4-3
factory/combo_generator.py	키워드 카르테시안 곱 생성	Fig 4-3

Table 7-5. 핵심 파일 레퍼런스 — 파일 경로, 역할, 관련 다이어그램 매핑

7.6 Complete Data Flow: 하나의 포스트가 만들어지는 전 과정

지금까지의 정적/동적 UML을 종합하여, Campaign 설정부터 블로그 발행까지 데이터가 어떻게 흘러가는지 하나의 시나리오로 추적합니다. 이것은 Martin이 말한 "고수준 다이어그램으로 시스템을 머릿속에 묶어라"의 실제 적용입니다.

단계	행위자	입력	출력	관련 Fig
1. 키워드 조합 생성	ComboGenerator	CampaignSlot (region, subject) + SpacingRule	Combination 행 (keyword_ids + spacing_rule_id)	Fig 4-3
2. 배치 배분	BatchDispatcher	미사용 Combination + 가용 Account	Batch + BatchItem (40개씩 배분)	Fig 4-3
3. Spec 조립	job_builder. build_posting_spec()	Combination ORM + Layout + Presets	PostingSpec (frozen dataclass)	Fig 4-1
4. Spec 검증	SpecValidator .validate()	PostingSpec	None 또는 ValueError	Fig 5-1
5. 콘텐츠 생성	ContentBuilder .build()	PostingSpec	_PostContent (title + steps + schedule)	Fig 5-2
6. 에디터 실행	JobRunner ._execute()	_PostContent + BlogEditor	발행된 블로그 포스트	Fig 5-1
7. 결과 기록	BatchWorker ._process_item()	실행 결과	ItemResult (success/error)	Fig 5-3

Table 7-6. 하나의 포스트가 만들어지는 7단계 데이터 흐름

이 7단계에서 단계 3이 두 레이어의 경계입니다. 1-2는 순수 Factory 영역(DB 조작), 4-6은 순수 Automator 영역(콘텐츠 생성 + 브라우저 제어), 7은 다시 Factory 영역(결과 기록)입니다. PostingSpec은 단계 3에서 생성되어 단계 4-6에서 소비됩니다.

구체적인 예를 들어보겠습니다. Campaign에 region=[강남, 서초], subject=[수학, 영어], SpacingRule 1개가 있다면:

- 단계 1: ComboGenerator가 $2 \times 2 \times 1 = 4$ 개의 Combination을 생성합니다. 예: (강남, 수학, rule1), (강남, 영어, rule1), (서초, 수학, rule1), (서초, 영어, rule1).
- 단계 2: BatchDispatcher가 가용 Account 'user_a'에 4개를 모두 배분합니다 (40개 미만이므로 1개 Batch).
- 단계 3: 첫 번째 item(강남, 수학)에 대해 build_posting_spec()이 호출됩니다. Campaign.title_template이 '{region} {subject} 과외 추천'이면, TitleOption.values = {region: '강남', subject: '수학'}이 됩니다.
- 단계 5: ContentBuilder가 Layout의 ParagraphBlock에 대해 SEO 프롬프트를 생성합니다. keyword='강남 수학'이 첫 단락에서는 첫 문장 안에 3회, 마지막 단락에서는 자연스럽게 2회 포함되도록 지시합니다.
- 단계 6: BlogEditor.open() -> write_title('강남 수학 과외 추천') -> insert_text(AI 생성 단락) -> publish().

7.7 자주 묻는 질문 (FAQ)

Q: factory가 automator의 block_factory를 직접 import하는 것은 DIP 위반 아닌가요?

A: 엄격한 DIP 관점에서는 맞습니다. 하지만 block_factory는 순수 함수(type string -> Block dataclass)로, 외부 의존성이 없는 유틸리티입니다. Martin도 모든 의존성을 역전할 필요는 없다고 합니다. 비용이 이익을 초과하는 경우에만 역전합니다. block_factory는 상태가 없고 테스트가 쉬우므로 직접 import이 합리적인 트레이드오프입니다.

Q: BlogEditor에 insert_text 하나로 heading, list, quote를 모두 처리하는 것은 너무 거칠지 않나요?

A: 의도적인 설계입니다. 이것은 ISP(Interface Segregation)와 미래 유연성의 균형입니다. 현재 타겟 플랫폼(네이버 블로그)에서는 SmartEditor가 내부적으로 구분하므로, BlogEditor ABC 수준에서는 'insert_text'로 충분합니다. 나중에 WordPress 같은 플랫폼에서 구분이 필요하다면, 그 구체 에디터 내부에서 step의 타입을 검사하면 됩니다.

Q: ContentContext가 mutable인 이유는? frozen dataclass 철학과 모순되지 않나요?

A: ContentContext는 handler 간에 상태를 공유하기 위한 의도적 mutable 객체입니다. paragraph_index가 각 ParagraphHandler 호출마다 증가해야 하기 때문입니다. frozen dataclass는 '레이어 간 계약'에 적용하고, '레이어 내부의 실행 상태'에는 mutable를 허용합니다. PHP의 Request(immutable) vs Controller state(mutable) 구분과 같은 논리입니다.

Q: 왜 BlockHandler를 singleton 인스턴스로 레지스트리에 등록하나요?

A: 모든 handler가 stateless이기 때문입니다. ParagraphHandler()는 인스턴스 변수를 가지지 않으며, 모든 상태는 ContentContext를 통해 전달됩니다. 따라서 매번 새로운 인스턴스를 만들 필요 없이 하나의 인스턴스를 재사용합니다. 이것은 Flyweight 패턴의 간단한 적용입니다.

Q: 테스트에서 mock.patch 대신 직접 test double을 쓰는 이유는?

A: Port ABC가 의존성 주입의 경계를 제공하기 때문에, mock.patch로 모듈 수준 바인딩을 교체할 필요가 없습니다. constructor injection으로 StubTextGenerator를 직접 넣으면 됩니다. 이 방식은 테스트가 구현 세부사항(import 경로)에 의존하지 않으므로 리팩토링에 더 견고합니다. Martin의 TDD 원칙과 일치합니다.

7.8 참고 자료

- Robert C. Martin, UML for Java Programmers (Prentice Hall, 2002) — 이 교재의 이론적 기반. 특히 Ch.2(Working with Diagrams), Ch.3(Class Diagrams), Ch.4(Sequence Diagrams), Ch.6(Principles of OOD).
- Robert C. Martin, Clean Code (Prentice Hall, 2008) — 이 코드베이스의 네이밍 컨벤션, 함수 크기, 클래스 책임 분리의 가이드라인.
- Robert C. Martin, Clean Architecture (Prentice Hall, 2017) — Factory/Automater 레이어 분리와 DIP 적용의 이론적 근거.
- Gamma et al., Design Patterns (Addison Wesley, 1994) — Command(PostStep), Strategy(BlockHandler), Factory Method(create_block) 패턴의 원전.
- automator/contracts.py 및 automator/ports.py — 이 교재의 모든 다이어그램의 '진짜' 소스. 다이어그램과 코드가 불일치하면, 코드가 진실입니다.

Martin의 마지막 조언

"UML은 도구이지 목적이 아니다. 도구로서 절제하여 사용하면 큰 이익을 준다. 남용하면 많은 시간을 낭비하게 된다. UML을 쓸 때는 작게 생각하라." 이 교재도 같은 원칙으로 작성되었습니다. 부족하다 느끼면 코드를 읽으세요.

전체 의존성 방향 요약



Fig 7-1. 전체 의존성 방향 — 양쪽 레이어 모두 Contracts를 향함 (DIP)

이 교재는 Martin의 조언을 따라 "작게 유지"했습니다. 25-200페이지 문서화 권장 범위의 하단에 맞추었으며, 세부 사항은 코드베이스의 docstring과 테스트 코드를 참조하세요. "다이어그램 위에서 코드를 떠올릴 수 없다면, 코드를 먼저 읽으세요."